کتابخانهٔ قابل‌حمل سنسور دما

علیرضا خواجه‌وندی

راهنمای پیاده‌سازی و آموزش گام‌به‌گام

STM32F107VCT6 / PB10

2026-10-10

صفحات مجازی علیرضا خواجه‌وندی

وب‌سایت · گیت‌هاب · لینکدین

یوتیوب · تلگرام · اینستاگرام

فهرست مطالب

۱ هدف، محدوده و نقطهٔ شروع ۴

۱.۱ اطلاعات برد و حدود تأیید ۴

۲.۱ خروجی این مرحله ۴

۲ اتصال و شناخت سنسور ۴

۱.۲ اتصال سه‌سیمه ۵

۲.۲ تغذیه و مقدار شروع ۵

۳.۲ اتصال پروگرامر ۵

۳ معماری و انتقال به میکروی دیگر ۵

۱.۳ قرارداد لایهٔ سخت‌افزار ۵

۲.۳ حافظه و هم‌زمانی ۶

۴ معرفی توابع و مثال‌های کاربردی ۶

۱.۴ شروع با نمونهٔ آماده ۶

۲.۴ راه‌اندازی باس ۶

۳.۴ گرفتن و بررسی آدرس ۷

۴.۴ خواندن کامل یک سنسور ۷

۵.۴ جداکردن شروع تبدیل از خواندن ۷

۶.۴ معرفی بافرها به مدیر آرایه ۸

۷.۴ پردازش و دریافت خروجی آرایه ۹

۸.۴ تعویض دستی در خانهٔ مشخص ۹

۹.۴ تنظیم و تأیید رزولوشن ۱۰

۱۰.۴ توابع خام و معنی خطاها ۱۰

۵ حفظ خانه‌ها و تعویض سنسور ۱۱

۱.۵ مثال تعویض روشن ۱۱

۲.۵ ابهام و انتخاب دستی ۱۱

۳.۵ کشف ناقص و ظرفیت ۱۱

۴.۵ ماندگاری نگاشت ۱۱

۶ زمان‌بندی، بهینگی و خطاها ۱۲

۱.۶ هزینهٔ واقعی اجرا ۱۲

۲.۶ اعتبارسنجی پیش از استفاده ۱۲

۳.۶ عیب‌یابی ۱۲

۷ ساخت، پروگرام و تولید مجدد ۱۳

۱.۷ پلتفرم توسعه ۱۳

۲.۷ توسعه با کیل ۱۳

۳.۷ بازسازی بدون ازبین‌رفتن کتابخانه ۱۳

۴.۷ پروژهٔ بعدی ۱۴

۸ اعتبارسنجی و پذیرش روی برد ۱۴

۱.۸ شواهد خودکار ۱۴

۲.۸ مراحل آزمون برد ۱۴

۳.۸ نگهداری مستندات ۱۵

۱ هدف، محدوده و نقطهٔ شروع

هر سنسور یا آی‌سی را به شکل پروژه‌ای مستقل سازمان‌دهی می‌کنم؛ فایل‌های برنامه، کتابخانه، پشتیبانی میکرو، تنظیمات ابزارها و مستندات داخل همان پوشه هستند. پروژهٔ اول برای سنسور دما روی پایهٔ PB10 و میکروی STM32F107VCT6 است؛ خود کتابخانه به این میکرو وابسته نیست.

پروژهٔ کپی‌شده شامل نمایشگر، رله، اندازه‌گیری الکتریکی، چند گذرگاه دما و ذخیره‌سازی تنظیمات بود. این امکانات از پروژهٔ جدید حذف شده‌اند. فقط پایهٔ توسعه و بخش‌های بررسی‌شدهٔ پروتکل حفظ شده‌اند. قالب خام هیچ منطق سنسوری ندارد.

۱.۱ اطلاعات برد و حدود تأیید

تنظیمات اولیه کریستال خارجی ۸ مگاهرتز، ضریب ۹، هستهٔ ۷۲ مگاهرتز و تغذیهٔ طراحی ۳٫۳ ولت را نشان می‌دهند. ظرفیت میکرو ۲۵۶ کیلوبایت فلش و ۶۴ کیلوبایت حافظهٔ داده است. [۱] این اطلاعات از فایل‌ها استخراج شده‌اند؛ اندازه‌گیری مدار، تغذیه و موج پایه انجام نشده است. پایهٔ سنسور مطابق اعلام کاربر انتخاب شده و امکانات قدیمی نمایشگر و رله در این نسخه به کار نمی‌روند.

۲.۱ خروجی این مرحله

خواندن یک آدرس یا آرایهٔ آدرس‌ها، آرایهٔ دما و وضعیت هم‌ترتیب، تبدیل غیرمسدودکننده و جایگزینی با حفظ خانه پیاده شده‌اند. حافظهٔ پویا وجود ندارد. تغذیهٔ خارجی لازم است؛ تغذیهٔ انگلی، ماندگاری نگاشت و انتقال سخت‌افزاری توسعه‌های آینده هستند.

۲ اتصال و شناخت سنسور

دیتاشیت اصلی سنسور، آدرس هشت‌بایتی و حافظهٔ موقت نه‌بایتی را تعریف می‌کند. [۲] بایت نخست آدرس کد خانوادهٔ 0x28 است، شش بایت شناسه و بایت آخر کنترل خطاست. دو بایت نخست حافظهٔ موقت، دمای علامت‌دار با واحد یک‌شانزدهم درجه را نگه می‌دارند. بایت چهارم رزولوشن را تعیین می‌کند و بیت‌های تعریف‌نشده در رزولوشن پایین باید حذف شوند. بایت آخر نیز کنترل صحت داده است. رزولوشن با دقت مطلق سنسور یکسان نیست.

۱.۲ اتصال سه‌سیمه

داده را به پایهٔ سنسور میکرو، تغذیه را به ۳٫۳ ولت و زمین را به زمین مشترک وصل کنید. مقاومت بالاکش معمولاً ۴٫۷ کیلواهم بین داده و ۳٫۳ ولت قرار می‌گیرد. ترتیب پایه را از دیتاشیت همان بسته‌بندی بررسی کنید؛ رنگ کابل معیار قطعی نیست. برای کابل بلند یا بار خازنی زیاد، مقاومت و توپولوژی باید با اندازه‌گیری موج ارزیابی شوند. پایهٔ میکرو خروجی open-drain است؛ نوشتن یک یعنی رهاکردن خط، نه اعمال فعال ولتاژ بالا.

۲.۲ تغذیه و مقدار شروع

این نسخه بالاکش قوی ندارد و تغذیهٔ انگلی را پشتیبانی نمی‌کند. اتصال تغذیهٔ خارجی الزامی است. مقدار اولیهٔ رجیستر دما ۸۵ درجه است، ولی همین عدد می‌تواند دمای واقعی باشد. روش آرایه‌ای پیش از خواندن تبدیل را شروع می‌کند و عدد ۸۵ را صرفاً به دلیل مقدارش رد نمی‌کند. برای خواندن مستقیم، مسئولیت انجام تبدیل با برنامه است. اگر سنسور وسط تبدیل ریست شود، تنها مقدار دما تازگی داده را اثبات نمی‌کند؛ کاربرد حساس به تشخیص‌های اضافی نیاز دارد.

۳.۲ اتصال پروگرامر

پایه‌های PA13/SWDIO و PA14/SWCLK را همراه زمین و مرجع ولتاژ هدف به پروگرامر وصل کنید. برد را از تغذیهٔ مناسب خودش روشن کنید. فعال‌بودن دیباگر به معنی صحت اتصال سنسور نیست.

۳ معماری و انتقال به میکروی دیگر

فایل‌های تولیدشونده در Core و لایهٔ برد در Application قرار دارند. پروتکل عمومی و فرمان‌های سنسور در Libraries هستند. فایل‌های سازنده در Drivers برای هر دو ابزار مشترک‌اند. [۳] فایل شروع اسمبلی کامپایلرهای مختلف جداست و ابزارهای همگام‌سازی و آزمون در پوشه‌های مستقل قرار دارند.

۱.۳ قرارداد لایهٔ سخت‌افزار

ساختار OneWire_Bus یک اشاره‌گر زمینه و هایcallback پایین‌کشیدن، رهاکردن، خواندن و تأخیر میکروثانیه‌ای دریافت می‌کند. تأخیر باید با وقفهٔ بسته هم اجرا شود. توابع ورود و خروج ناحیهٔ حساس اختیاری ولی جفت هستند؛ ورود وضعیت قبلی وقفه را برمی‌گرداند و خروج دقیقاً همان وضعیت را بازیابی می‌کند. حذف هر دو فقط وقتی مجاز است که محیط برنامه زمان‌بندی را به روش دیگری تضمین کند.

در نمونهٔ فعلی شمارندهٔ سیکل DWT پس از تنظیم کلاک فعال می‌شود. نوشتن رجیسترهای GPIO نیاز به تغییر

مکرر حالت پایه ندارد. اختلاف شمارنده برای تأخیرهای کوتاه با سرریز سازگار است. میکروهای فاقد شمارندهٔ قابل‌استفاده باید تأخیر را با تایمر پیاده کنند؛ هدرهای کتابخانه هیچ وابستگی به STM۳۲ ندارند.

۲.۳ حافظه و هم‌زمانی

برنامه ساختار مدیریت، دو آرایهٔ جداگانهٔ آدرس، آرایهٔ دما و وضعیت را با ظرفیت یکسان تخصیص می‌دهد. همهٔ این بافرها باید تا پایان کار زنده بمانند. تخصیص پویا و نمونهٔ سراسری اجباری وجود ندارد. دسترسی به یک باس باید سری شود؛ از وقفه فراخوانی نکنید و هنگام تبادل کلاک را عوض نکنید. در سیستم عامل، قفل دسترسی باید کل تراکنش را پوشش دهد و فقط محدود به یک callback نباشد.

۴ معرفی توابع و مثال‌های کاربردی

۱.۴ شروع با نمونهٔ آماده

تابع App_Init() را پس از راه‌اندازی HAL، کلاک و GPIO فقط یک بار و تابع App_Process() را مرتب در حلقهٔ اصلی اجرا کنید. تعداد خانه‌ها را با APP_SENSOR_COUNT تعیین کنید؛ برای یک سنسور مقدار یک کافی است. آرایه‌های آدرس، دما و وضعیت با اندیس یکسان به یک سنسور اشاره می‌کنند. اندیس صفر اولین سنسور است. فقط وضعیت موفق اجازهٔ استفاده از دما را می‌دهد.

برای میکروی دیگر ابتدا هایcallback پایه و زمان را معرفی کنید. کتابخانه خواندن کامل مسدودکننده، شروع و خواندن جداگانه و رابط آرایه‌ای با انتظار آزاد دارد. رابط آرایه‌ای با یک سنسور نیز قابل‌استفاده است.

۲.۴ راه‌اندازی باس

تابع OneWire_Init ساختار مقصد باس و تنظیم هاcallback را می‌گیرد. اگر توابع الزامی و جفت حفاظت وقفه معتبر باشند، تنظیمات را کپی و خط را رها می‌کند و مقدار درست می‌دهد. این تابع سنسور را کشف نمی‌کند. قرارداد هاcallback در فصل معماری و نمونهٔ برد در Application/Src/app.c آمده است.

```
OneWire_Bus bus;
/* callbacks contains your board's GPIO/time functions. */
bool bus_ok = OneWire_Init(&bus, &callbacks);
```

اگر اینیت ناموفق بود ادامه ندهید. ساختار باس باید تا پایان کار مدیر زنده بماند. مثال‌های بعدی فرض می‌کنند باس با موفقیت راه‌اندازی شده است.

۳.۴ گرفتن و بررسی آدرس

تابع DS18B20_ReadRom باس و آرایهٔ خروجی هشت‌بایتی را می‌گیرد و وضعیت عملیات را می‌دهد. فقط وقتی دقیقاً یک سنسور متصل است از آن استفاده کنید. تابع DS18B20_IsValidRom خانواده و CRC آدرس را بررسی می‌کند ولی حضور فعلی آن سنسور را اثبات نمی‌کند.

```
uint8_t rom[DS18B20_ROM_SIZE];
DS18B20_Status result = DS18B20_ReadRom(&bus, rom);
bool address_ok = (result == DS18B20_STATUS_OK);
```

برای چند سنسور از کشف خودکار مدیر یا توابع OneWire_SearchFirst و OneWire_SearchNext با ساختار وضعیت متعلق به برنامه استفاده کنید. هر جست‌وجوی موفق یک آدرس با CRC معتبر می‌دهد؛ خانوادهٔ سنسور را هم کنترل کنید. ترتیب جست‌وجو شمارهٔ مکان نیست.

۴.۴ خواندن کامل یک سنسور

تابع DS18B20_ReadTemperatureBlocking باس، آدرس، بودجهٔ اسمی انتظار میلی‌ثانیه‌ای و اشاره‌گر خروجی را می‌گیرد. تبدیل را شروع می‌کند، منتظر می‌ماند و دمای تأییدشده را می‌خواند. فقط موفقیت خروجی را تغییر می‌دهد. برای رزولوشن نامعلوم بودجهٔ ۱۰۰۰ میلی‌ثانیه مناسب است؛ سربار تبادل به بودجهٔ اسمی اضافه می‌شود.

```
float temperature_c = 0.0f;
if (address_ok) {
    result = DS18B20_ReadTemperatureBlocking(
        &bus, rom, 1000U, &temperature_c);
}
bool temperature_valid = (result == DS18B20_STATUS_OK);
```

این کد ادامهٔ مثال آدرس است. فقط با اعتبار درست از خروجی استفاده کنید؛ صفر اولیه یا عدد قبلی اندازه‌گیری موفق نیست. این تابع حدود مدت تبدیل برنامه را متوقف می‌کند. برای انجام کارهای دیگر حلقه، روش بعدی یا مدیر آرایه را انتخاب کنید.

۵.۴ جداکردن شروع تبدیل از خواندن

تابع DS18B20_StartConversion فرمان تبدیل را برای آدرس می‌فرستد؛ آدرس تهی یعنی فرمان عمومی باس. موفقیت یعنی ارسال فرمان، نه پایان اندازه‌گیری. تابع DS18B20_IsConversionReady آماده‌بودن را بررسی می‌کند. تابع DS18B20_ReadTemperature نتیجهٔ تبدیل انجام‌شده را با کنترل CRC و بازه می‌خواند؛ تبدیل تازه آغاز نمی‌کند و فقط در موفقیت خروجی را تغییر می‌دهد.

در STM۳۲ از tick سیستم و در میکروی دیگر از ساعت میلی‌ثانیه‌ای یکنواخت استفاده کنید. قطعهٔ زیر را یک بار اجرا و متغیرهای وضعیت را بین تکرارهای حلقه حفظ کنید:

```
bool waiting = false;
uint32_t deadline = 0;
temperature_valid = false;
result = DS18B20_StartConversion(&bus, rom);
if (result == DS18B20_STATUS_OK) {
    deadline = HAL_GetTick() + 760U; /* After command. */
    waiting = true;
}
```

قطعهٔ بعدی را مرتب در حلقهٔ اصلی اجرا کنید:

```
if (waiting &&
    (int32_t)(HAL_GetTick() - deadline) >= 0) {
    waiting = false;
    result = DS18B20_IsConversionReady(&bus)
        ? DS18B20_ReadTemperature(&bus, rom, &temperature_c)
        : DS18B20_STATUS_TIMEOUT;
    temperature_valid = (result == DS18B20_STATUS_OK);
}
/* Other main-loop work continues while waiting. */
```

میان شروع تبدیل و بررسی آماده‌بودن باس را ریست نکنید و جست‌وجو یا فرمان دیگری نفرستید. مدیر آرایه این توالی را انجام می‌دهد و معمولاً ساده‌تر است.

۶.۴ معرفی بافرها به مدیر آرایه

تابع DS18B20_ArrayInit مدیر، باس، آرایهٔ آدرس قابل‌تغییر، بافر جداگانهٔ کشف، آرایهٔ دما، آرایهٔ وضعیت و تعداد را دریافت می‌کند. همهٔ بافرها همین ظرفیت را دارند و تا پایان کار زنده می‌مانند. اشاره‌گر یا تعداد نامعتبر، آدرس تکراری و آدرس غیرصفر نامعتبر نتیجهٔ نادرست می‌دهند. آدرس صفر یعنی کشف خودکار.

```
#define N 4U /* Use 1U for one sensor. */
static DS18B20_Array sensors;
static DS18B20_Rom addresses[N] = {{0}};
static DS18B20_Rom discovered[N];
static float temperatures[N];
static DS18B20_Status status[N];
bool initialized = DS18B20_ArrayInit(
    &sensors, &bus, addresses, discovered,
    temperatures, status, N);
if (initialized) sensors.sample_interval_ms = 250U;
```

اینیت را فقط یک بار انجام دهید. فاصلهٔ تنظیم‌شده، استراحت پس از چرخه است و مدت تبدیل نیست. برای آدرس معلوم آرایه را پیش از اینیت پر کنید؛ ترتیب بایت‌ها خانواده، شش بایت شناسه و CRC است. مدیر می‌تواند آدرس را عوض کند؛ آرایه را ثابت تعریف نکنید. بافر ایستا از مصرف زیاد پشته جلوگیری می‌کند.

۷.۴ پردازش و دریافت خروجی آرایه

تابع DS18B20_ArrayProcess مدیر و tick فعلی را می‌گیرد. مرتب صدا بزنید. نتیجهٔ درست یعنی چرخهٔ کامل پایان یافته، حتی اگر بعضی خانه‌ها خطا داشته باشند. نتیجهٔ نادرست یعنی هنوز چرخه کامل نشده و نشانهٔ خرابی نیست. وضعیت هر خانه را جدا بررسی کنید؛ ترتیب خروجی همان ترتیب آرایهٔ آدرس است.

```
static float latest_c[N];
static bool valid[N];
if (initialized &&
    DS18B20_ArrayProcess(&sensors, HAL_GetTick())) {
    for (size_t i = 0; i < N; ++i) {
        valid[i] = (status[i] == DS18B20_STATUS_OK);
        if (valid[i]) latest_c[i] = temperatures[i];
    }
}
```

این تصویر اختیاری، عددهای موفق قبلی را نگه می‌دارد ولی خانهٔ ناموفق را نامعتبر می‌کند. فقط خانهٔ معتبر را برای نمایش یا کنترل استفاده کنید. خروجی خود کتابخانه هنگام تبدیل و خطا عدد نامعتبر است.

۸.۴ تعویض دستی در خانهٔ مشخص

تابع DS18B20_ArrayReplace مدیر، اندیس مقصد از صفر و آدرس معتبر جدید را می‌گیرد. موفقیت نگاشت را تغییر و دمای قبلی را نامعتبر می‌کند. شکست می‌تواند از اندیس یا آدرس نامعتبر، آدرس استفاده‌شده در خانهٔ دیگر یا آزادنبودن مدیر باشد. آدرس جدید را از جست‌وجوی کامل یا اتصال تک‌سنسوری جداگانه بگیرید؛ روی باس چندسنسوری آدرس را با ROM Read نخوانید.

```
/* replacement_rom was obtained and validated earlier. */
if (sensors.phase == 0U) {
    bool replaced = DS18B20_ArrayReplace(
        &sensors, 1U, replacement_rom); /* Second slot. */
    /* Check replaced, then wait for a valid new sample. */
}
```

مثال به دو خانه یا بیشتر نیاز دارد و باید از همان task پردازش اجرا شود. تغییر آدرس دمای تازه نمی‌دهد؛ منتظر نمونهٔ معتبر بعدی باشید. نگاشت بدون ذخیره‌سازی برنامه پس از ریست باقی نمی‌ماند. تعویض خودکار در فصل

بعد توضیح داده شده است.

۹.۴ تنظیم و تأیید رزولوشن

تابع DS18B20_WriteScratchpad باس، آدرس، آلارم بالا، آلارم پایین و رزولوشن را می‌گیرد. مثال، رزولوشن ۱۰ بیت و حدود ۳۰ و ۱۰ درجه را تنظیم می‌کند. تنظیم موقتی است و در EEPROM کپی نمی‌شود. موفقیت ارسال به‌تنهایی تأیید سنسور آدرس‌دار نیست؛ با توابع خواندن حافظه و دریافت رزولوشن نتیجه را بررسی کنید:

```
result = DS18B20_WriteScratchpad(
    &bus, rom, 30, 10, DS18B20_RESOLUTION_10_BIT);
uint8_t pad[DS18B20_SCRATCHPAD_SIZE];
if (result == DS18B20_STATUS_OK)
    result = DS18B20_ReadScratchpad(&bus, rom, pad);
bool configured = (result == DS18B20_STATUS_OK) &&
    DS18B20_GetResolution(pad) == DS18B20_RESOLUTION_10_BIT &&
    pad[2] == 30U && pad[3] == 10U;
```

پیش از نمونه‌گیری یا در زمان آزاد مدیر تنظیم کنید. مدیر همچنان انتظار حداکثر ۱۲ بیت را دارد. تابع DS18B20_GetConversionTimeMs برای ۹ تا ۱۲ بیت به‌ترتیب ۹۴، ۱۸۸، ۳۷۵ و ۷۵۰ میلی‌ثانیه می‌دهد؛ در زمان‌بندی اختصاصی حاشیهٔ مناسب را اضافه کنید.

۱۰.۴ توابع خام و معنی خطاها

تابع خواندن حافظهٔ موقت نه بایت را اعتبارسنجی می‌کند. فقط پس از موفقیت آن تابع DS18B20_DecodeRaw مقدار علامت‌دار را استخراج و بیت‌های تعریف‌نشده را حذف می‌کند؛ تابع DS18B20_RawToCelsius آن را به درجه تبدیل می‌کند. برای استفادهٔ معمول تابع خواندن دما را انتخاب کنید که بازهٔ معتبر را نیز کنترل می‌کند.

موفقیت اجازهٔ استفاده می‌دهد و مشغول یعنی انتظار. نبود دستگاه یعنی حضور در ریست دیده نشده؛ خطای CRC یعنی دادهٔ غیرقابل‌استفاده؛ پایان مهلت یعنی تبدیل آماده نشده؛ خروج از بازه یعنی مقدار ناممکن؛ ورودی نامعتبر نیازمند اصلاح آدرس یا فراخوانی است. سنسور آدرس‌دار قطع‌شده می‌تواند با حضور سنسور دیگر خطای CRC بدهد. وضعیت را معیار بگیرید، نه صرفاً عدد دما. توابع ریست، بیت و بایت اجزای پروتکل هستند و معمولاً لایهٔ سنسور آن‌ها را صدا می‌زند؛ پیاده‌سازی آن‌ها در سورس قابل مطالعه است و در انتهای گزارش تکرار نمی‌شود.

۵ حفظ خانه‌ها و تعویض سنسور

ترتیب جست‌وجوی ROM از داوری بیت‌ها حاصل می‌شود و شمارهٔ مکان فیزیکی نیست. آدرس‌های موجود با هر هشت بایت تطبیق داده می‌شوند؛ تغییر ترتیب جست‌وجو خروجی آرایه را جابه‌جا نمی‌کند. وقتی سنسور تعریف‌شدهٔ غایب وجود ندارد، آدرس جدید اولین خانهٔ خالی را می‌گیرد.

۱.۵ مثال تعویض روشن

فرض کنید آرایه شامل سه آدرس A، B و C باشد. B قطع و D متصل می‌شود. جست‌وجوی کامل A، C و D را می‌یابد؛ یک آدرس غایب و یک آدرس ناشناخته داریم. نگاشت به A، D و C تبدیل می‌شود و خانهٔ دوم حفظ می‌شود. کشف تقریباً هر دو ثانیه انجام می‌شود و سپس نمونه‌گیری صورت می‌گیرد؛ تعویض آنی نیست. مقدار نامعتبر یا درحال تبدیل به شکل عدد معتبر قبلی ارائه نمی‌شود.

۲.۵ ابهام و انتخاب دستی

اگر دو سنسور حذف و دو سنسور اضافه شوند، نرم‌افزار محل فیزیکی آن‌ها را نمی‌داند. نگاشت حفظ می‌شود. سنسورها را یکی‌یکی عوض کنید یا با تابع DS18B20_ArrayReplace خانهٔ مشخصی را در فاز صفر انتخاب کنید. آدرس استفاده‌شده در خانهٔ دیگر رد می‌شود. این کار را از همان task پردازش و بین دو چرخه انجام دهید؛ وسط تبدیل مجاز نیست.

۳.۵ کشف ناقص و ظرفیت

خطای کنترل صحت، پاسخ ناممکن بیت‌ها یا نبود حضور، وضعیت جست‌وجو را بازنشانی می‌کند. نگاشت خودکار فقط پس از شمارش کامل مجاز است. بیشتر بودن تعداد سنسورها از ظرفیت بافر پرچم سرریز را فعال و تغییر خودکار را متوقف می‌کند. تعداد خانه‌ها را برای کل سنسورهای باس کافی انتخاب کنید. سنسور موجود ولی از نظر الکتریکی غیرقابل‌خواندن همچنان خطاست؛ تشخیص مکان بدون آدرس یا باس جدا ممکن نیست.

۴.۵ ماندگاری نگاشت

آدرس‌ها در RAM هستند و با ریست باقی نمی‌مانند. برای حفظ برچسب‌ها از آدرس‌های معلوم استفاده یا ذخیره‌سازی مستقل با CRC و حفاظت قطع برق اضافه کنید. کتابخانه صفحهٔ Flash رزرو نمی‌کند. ذخیره‌سازی را هر چرخه انجام ندهید؛ سیاست ماندگاری متناسب با هر پروژه باید در لایهٔ برنامه انتخاب شود.

۶ زمان‌بندی، بهینگی و خطاها

مدیر فازهای انتظار آزاد، جست‌وجو، شروع تبدیل، انتظار تبدیل، خواندن و پایان را دارد. هر فراخوانی یک مرحلهٔ کوچک انجام می‌دهد. جست‌وجو یک آدرس و خواندن یک حافظهٔ موقت را پردازش می‌کند؛ انتظار تا موعد زمانی فوراً برمی‌گردد. تبدیل همهٔ سنسورهای با تغذیهٔ خارجی با فرمان عمومی آغاز می‌شود. موعد ۷۶۰ میلی‌ثانیه زمان فرمان و گردکردن tick را کنار حداکثر ۷۵۰ میلی‌ثانیه لحاظ می‌کند. تفاضل علامت‌دار با سرریز سازگار است، به شرط آنکه فاصلهٔ سرویس کمتر از نصف بازهٔ شمارندهٔ ۳۲ بیتی باشد.

۱.۶ هزینهٔ واقعی اجرا

انتظار تبدیل آزاد است ولی خود تبادل سنکرون است. یک جست‌وجو حدود ۱۵ میلی‌ثانیه و یک خواندن آدرس‌دار حدود ۱۲ میلی‌ثانیه زمان foreground مصرف می‌کند. چرخه با تعداد سنسور بزرگ‌تر طولانی‌تر می‌شود. وقفه‌ها در بازیابی خط و انتظار تبدیل اجرا می‌شوند. تأخیرهای حساس درخواست‌شده تقریباً ۷۰ میکروثانیه برای حضور، ۶۰ برای صفر، ۶ برای یک و ۱۳ برای خواندن هستند. سربار callback اضافه می‌شود؛ این عددها سقف اندازه‌گیری‌شده روی برد نیستند. مهلت‌های سخت کوتاه‌تر به انتقال سخت‌افزاری یا تایمر و DMA و بررسی موج نیاز دارند.

۲.۶ اعتبارسنجی پیش از استفاده

کنترل خطای آدرس و داده از چندجمله‌ای بازتاب‌یافتهٔ $0x8C$ استفاده می‌کند. بیت‌های ثابت تنظیم و بایت‌های رزروشده هم بررسی می‌شوند تا پاسخ تمام‌صفر که CRC صفر معتبری دارد رد شود. بیت‌های تعریف‌نشدهٔ رزولوشن پایین حذف و بازهٔ منفی ۵۵ تا مثبت ۱۲۵ درجه کنترل می‌شود. شروع تبدیل خروجی‌ها را NAN و وضعیت را BUSY می‌کند. تنها موفقیت وضعیت OK دارد؛ در خطا مقدار نامعتبر باقی می‌ماند. برای تشخیص NAN از برابری استفاده نکنید؛ وضعیت یا تابع استاندارد isnan را بررسی کنید.

۳.۶ عیب‌یابی

نبود سنسور: تغذیه، زمین، مقاومت و پایه را بررسی کنید. خطای :CRC نویز، کابل، تأخیر یا سنسور قطع‌شده را بررسی کنید؛ حضور سنسور دیگر می‌تواند ریست را موفق کند ولی خواندن آدرس غایب خطا بدهد. پایان‌نیافتن تبدیل: تغذیه و خط پایین را بررسی کنید. ورودی نامعتبر: CRC آدرس، تعداد، عمر بافر و تکرار را کنترل کنید. دمای ۸۵: تکمیل تبدیل و ریست تغذیه را بررسی کنید ولی واقعی‌بودن مقدار را رد نکنید. خط پایین پیش از ریست یا پس از پنجرهٔ حضور نیز رد می‌شود.

پس از هر چرخه فاصلهٔ آزاد پیش‌فرض ۲۵۰ میلی‌ثانیه قرار دارد. فیلد $\mathrm{sample_interval_ms}$ پیش از پردازش قابل‌تنظیم است و تکرار تلاش در نبود سنسور را محدود می‌کند. موعد تبدیل در فراخوانی بعد از ارسال فرمان

ثبت می‌شود تا وقفهٔ زمان ارسال نتواند مدت انتظار را کوتاه کند.

۷ ساخت، پروگرام و تولید مجدد

۱.۷ پلتفرم توسعه

PlatformIO را نصب و پوشهٔ مستقل پروژه را باز کنید. نسخهٔ پلتفرم مشخص شده و فایل‌های سازنده از داخل پروژه ساخته می‌شوند. نام برد ارزیابی فقط مشخصات میکرو و پروگرامر را فراهم می‌کند و به معنی یکسان‌بودن برد سفارشی نیست.

```
pio run
pio run -e stm32f107vc_stlink
pio run -t upload
pio run -e stm32f107vc_stlink -t upload
```

هر دو محیط منابع مشترک و لینک‌اسکریپت ۲۵۶ کیلوبایت فلش را دارند. نصب اولیه ابزارها به دانلود نیاز دارد ولی فایل پروژه از پوشهٔ همسایه نمی‌خواند.

۲.۷ توسعه با کیل

فایل پروژهٔ داخل MDK را باز کنید. پکیج میکرو و کامپایلر نسخهٔ شش را نصب کنید. پروگرامر SWD و الگوریتم فلش ۲۵۶ کیلوبایت را در تنظیمات دیباگ و ابزارها انتخاب و Build و Download کنید. آرایه‌های آدرس، دما و وضعیت و شمارندهٔ چرخه را در Watch ببینید. XML و عضویت فایل‌ها بررسی شده‌اند؛ ساخت واقعی Keil انجام نشده است.

۳.۷ بازسازی بدون ازبین‌رفتن کتابخانه

فایل تنظیم CubeMX را باز و کلاک، پایهٔ open-drain و حفظ کد کاربر را نگه دارید. کتابخانه و برنامه بیرون از Core هستند و include و فراخوانی‌ها در بلوک‌های کاربر قرار دارند. مولد ممکن است پروژهٔ Keil را بازنویسی کند؛ اسکریپت پس‌تولید عضویت فایل‌ها و مسیرها را برمی‌گرداند. اگر مسیر نسبی شناخته نشد، فایل cmd را در تنظیمات Actions User صریح انتخاب کنید. پس از هر تولید اجرا کنید:

```
python tools/sync_keil.py
python tools/check_project.py
pio run
```

همگام‌سازی تکرارپذیر است و فایل شروع، اندازهٔ فلش و منابع برنامه، کتابخانه و HAL را بازمی‌گرداند. مسیر Inc کتابخانهٔ تازه را به PlatformIO اضافه کنید. افزودن peripheral به بررسی هدر ،HAL وقفه و MSP هم نیاز دارد. تغییرات را مرور و Keil را نیز بسازید. تولید واقعی CubeMX در این محیط انجام نشده است.

۴.۷ پروژهٔ بعدی

کل قالب خام را کپی کنید. فقط کلاک، ،SWD فایل شروع، پشتیبانی سازنده و توابع خالی برنامه دارد. نام پروژه را در CubeMX تغییر دهید، sync را اجرا کنید و کتابخانه و پایهٔ قطعهٔ بعدی را اضافه کنید. وضعیت و گزارش دو زبان همان پروژه را بنویسید و وابستگی‌ها را داخل همان پوشه حفظ کنید.

۸ اعتبارسنجی و پذیرش روی برد

۱.۸ شواهد خودکار

ساخت PlatformIO نمونه و قالب خام موفق شده است. بررسی ساختاری مسیرهای محلی، عضویت فایل‌ها و شروع اسمبلی، بلوک‌های کاربر، تنظیم پایه و اندازهٔ فلش را کنترل می‌کند. آزمون‌ها C واقعی را کامپایل و روی شبیه‌ساز هسته اجرا می‌کنند.

```
python -m pip install unicorn
python tests/run_tests.py
python tools/check_project.py
```

یک مجموعه پاسخ پروتکل را شبیه‌سازی و ترتیب، تعویض، ابهام، کشف ناقص، سرریز، خطای ،CRC قطع سنسور، timeout، علامت دما، بازه و سرریز tick را بررسی می‌کند. مجموعهٔ دیگر کد واقعی باس را با نمونه‌های خط اجرا می‌کند و خطای حضور، بردار ،CRC جست‌وجوی ROM و تأخیرهای حساس درخواست‌شده را می‌سنجد. این آزمون‌ها اثبات صحت الکترونیک و زمان‌بندی آنالوگ برد نیستند.

۲.۸ مراحل آزمون برد

با پروگرامر انتخاب‌شده بسازید و پروگرام کنید. یک سنسور با تغذیهٔ خارجی وصل کنید و پس از چرخهٔ کامل آدرس، وضعیت و دمای محیط را ثبت کنید. سنسورها را یکی‌یکی اضافه و خانه‌های جدید را بررسی کنید. آدرس‌های معلوم را تنظیم و ترتیب پس از ریست را کنترل کنید. یک سنسور را قطع کنید؛ وضعیت باید خطا و دما نامعتبر شود. سنسور جدید باید پس از کشف در همان خانه قرار گیرد. دو تعویض هم‌زمان باید تا انتخاب دستی نگاشت قبلی را حفظ کنند. نبود مقاومت و خط پایین را با روش امن بررسی کنید. موج را زیر بار وقفه و دما را در چند نقطه با

مرجع مستقل مقایسه کنید. نتایج فیزیکی هنوز ثبت نشده‌اند.

۳.۸ نگهداری مستندات

وضعیت پروژه را پس از هر مرحله به‌روز کنید. بررسی سورس، ساخت کامپایلر، تولید واقعی و اندازه‌گیری برد را جداگانه ثبت کنید. دیتاشیت دانلودشده را داخل پروژه نگه ندارید؛ پیوند رسمی و فایل منابع کافی است. PDF نهایی گزارش در هر دو زبان همراه منبع قابل‌ویرایش در Git نگهداری و در GitHub منتشر می‌شود. پس از تغییر گزارش، هر دو PDF را دوباره بسازید و بررسی کنید. فونت دانلودشده، تصاویر بررسی و فایل‌های موقت ساخت محلی‌اند و وارد Git نمی‌شوند.

مراجع

[۱] STMicroelectronics، *documentation product STM۳۲F۱۰۷VC*، ۲۰۲۶، accessed ۰۱۰-۱۰-۲۰۲۶. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm۳۲f۱۰۷vc.html

[۲] Maxim / Devices Analog Integrated، *DS۱۸B۲۰ Programmable Resolution ۱-Wire Digital Thermometer*، ۲۰۱۹، rev. ۶، accessed ۰۱۰-۱۰-۲۰۲۶. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds۱۸b۲۰.pdf

[۳] STMicroelectronics، *STM۳۲CubeF۱ official firmware sources*، ۲۰۲۶، vendor sources. [Online]. Available: https://github.com/STMicroelectronics/STM۳۲CubeF۱ licenses. individual their retain

دربارهٔ نویسنده و راه‌های ارتباطی

علیرضا خواجه‌وندی.

وب‌سایت:

https://alirezaaies.github.io/

گیت‌هاب:

https://github.com/alirezaaies

لینکدین:

https://www.linkedin.com/in/alirezakhajehvandi

یوتیوب:

https://www.youtube.com/@AlirezaAIES

تلگرام:

https://t.me/AlirezaAIES

اینستاگرام:

https://www.instagram.com/alireza.aies